agentic-ai-azure-langgraph

Companion - LangGraph on Azure

Agentic AI on Azure - Enterprise Master Class | Handout (slides + notes)

Notes

Welcome. This deck covers 'agentic-ai-azure-langgraph' - Companion - LangGraph on Azure. Frame the problem and why it matters, then walk the class through the learning goal, the enterprise use case, the architecture/flow, the key concepts, a live run in VS Code, and the architect's takeaways. The repo runs locally with no Azure, so demo it live.

Slide 2 - Learning goal

- Build an agent as an explicit graph
- Use typed state, nodes & conditional edges
- Compare LangGraph to the Microsoft Agent Framework

Notes

[Learning goal] Walk through these talking points:
- Build an agent as an explicit graph
- Use typed state, nodes & conditional edges
- Compare LangGraph to the Microsoft Agent Framework
Ask the class: which of these ideas is new to you?

Slide 3 - Enterprise use case

- Support Triage Graph
- Same triage problem as Week 1, as a StateGraph
- Shows the 'path' of nodes visited

Notes

[Enterprise use case] Walk through these talking points:
- Support Triage Graph
- Same triage problem as Week 1, as a StateGraph
- Shows the 'path' of nodes visited
Tip: relate this to a regulated scenario your students know.

Slide 4 - Architecture / flow

- START -> classify -> (escalate | retrieve -> respond) -> END
- classify sets severity & category
- conditional edge routes urgent -> escalate
- respond optionally rephrased by Azure OpenAI

Notes

[Architecture / flow] Walk through these talking points:
- START -> classify -> (escalate | retrieve -> respond) -> END
- classify sets severity & category
- conditional edge routes urgent -> escalate
- respond optionally rephrased by Azure OpenAI
Demo: trace a single request through these steps live.

Slide 5 - Key concepts

- StateGraph: add_node, add_conditional_edges, compile
- Graph runs for real offline (nodes are Python)
- langchain-openai AzureChatOpenAI is optional

Notes

[Key concepts] Walk through these talking points:
- StateGraph: add_node, add_conditional_edges, compile
- Graph runs for real offline (nodes are Python)
- langchain-openai AzureChatOpenAI is optional
Open the named file in the repo while you explain each point.

Slide 6 - Run it (VS Code - no Azure needed)

- git clone the repo, then open it in VS Code
- python -m venv .venv -> activate it
- pip install -r requirements.txt
- uvicorn app.main:app --reload
- Open /docs and call POST /api/v1/triage
- Runs in MOCK mode - no Azure/keys needed

Notes

[Run it (VS Code - no Azure needed)] Walk through these talking points:
- git clone the repo, then open it in VS Code
- python -m venv .venv -> activate it
- pip install -r requirements.txt
- uvicorn app.main:app --reload
- Open /docs and call POST /api/v1/triage
- Runs in MOCK mode - no Azure/keys needed
Pause for questions before moving on.

Slide 7 - Architect's takeaways

- LangGraph shines when control flow is the hard part
- Branches, loops, checkpoints, human-in-the-loop
- Vendor-neutral; works with Azure OpenAI

Notes

[Architect's takeaways] Walk through these talking points:
- LangGraph shines when control flow is the hard part
- Branches, loops, checkpoints, human-in-the-loop
- Vendor-neutral; works with Azure OpenAI
Discussion: when would you NOT choose this approach?